

unidirectinal

```
syms x y z w t
f=x^2+y^2+z^2+w^2-2*x-4*y+6*z+8*w+15;
a=[1 1 1 1];
b=[-1 0 1 1];
st=a+t*b;
fst=subs(f,[x y z w],st);
ft=diff(fst,t);
t1=solve(ft==0,t)
```

t1 = -3

```
ftt=diff(fst,t,2);
if subs(ftt,t,t1)>0
X=subs(st,t1);
disp("The minimum value of the function is at :")
disp(X);
disp(" Minimum value =")
disp(subs(f,[x,y,z,w],X))
end
```

The minimum value of the function is at :

(4 1 -2 -2)

Minimum value =

0

evolutionary

```
f = @(X) (X(1) - 2).^2 + (X(2) - 3).^2;
X0 = [1,0];
D = [2, 2];
e = 0.8;
X_bar = X0;
NormD = sqrt(D(1)^2 + D(2)^2);
k=0;
while NormD > e
X1 = X_bar + [-D(1)/2, -D(2)/2];
X2 = X_bar + [ D(1)/2, -D(2)/2];
X3 = X_bar + [ D(1)/2, D(2)/2];
X4 = X_bar + [-D(1)/2, D(2)/2];
f0 = f(X_bar);
f1 = f(X1);
f2 = f(X2);
f3 = f(X3);
f4 = f(X4);
F = [f0, f1, f2, f3, f4];
[minimum, idx] = min(F);
switch idx
case 1
```

```

D = D / 2;
NormD = sqrt(D(1)^2 + D(2)^2);
case 2
X_bar = X1;
case 3
X_bar = X2;
case 4
X_bar = X3;
otherwise
X_bar = X4;
end
k=k+1;
fprintf('\n The minimum value after %d iteration is %.3f at
X=(%.3f,%.3f)',k,f(X_bar),X_bar(1),X_bar(2));
end

```

```

The minimum value after 1 iteration is 4.000 at X=(2.000,1.000)
The minimum value after 2 iteration is 2.000 at X=(3.000,2.000)
The minimum value after 3 iteration is 0.000 at X=(2.000,3.000)
The minimum value after 4 iteration is 0.000 at X=(2.000,3.000)
The minimum value after 5 iteration is 0.000 at X=(2.000,3.000)

```

```
disp('Optimal point:');
```

```
Optimal point:
```

```
disp(X_bar);
```

```
2    3
```

```
disp('Function value at optimal point:');
```

```
Function value at optimal point:
```

```
disp(f(X_bar));
```

```
0
```

hoke jeeves

```

f = @(X) (X(1)-1)^2 + (X(2)-2)^2;
X0 = [0, 0];
D = [0.5, 0.5];
a = 2;
E = 0.5;
k = 1;
X1 = exploratory(X0, D, f);
fprintf('Iteration %d: X = [%f, %f], f(X) = %f\n', k, X1(1), X1(2), f(X1));

```

```
Iteration 1: X = [0.500000, 0.500000], f(X) = 2.500000
```

```

k = k + 1;
while norm(D) > E
    X1p = 2 * X1 - X0;

```

```

X2 = exploratory(X1p, D, f);
if any(X1p ~= X2) || f(X2) < f(X1)
    X0 = X1;
    X1 = X2;
else
    D = D / a;
end
fprintf(' X = [%f, %f], f(X) = %f\n', X1(1), X1(2), f(X1));
k = k + 1;
end

```

```

X = [1.000000, 1.500000], f(X) = 0.250000
X = [1.000000, 2.000000], f(X) = 0.000000
X = [1.000000, 2.000000], f(X) = 0.000000
X = [1.000000, 2.000000], f(X) = 0.000000

```

```
fprintf('Final Solution: X = [%f, %f], f(X) = %f\n', X1(1), X1(2), f(X1));
```

```
Final Solution: X = [1.000000, 2.000000], f(X) = 0.000000
```

```

function [xmin] = exploratory(X0, D, f)
    d = D(1);
    f0 = f(X0);
    f_plus = f(X0 + [d, 0]);
    f_minus = f(X0 - [d, 0]);
    [fmin1, idx] = min([f0, f_plus, f_minus]);
    switch idx
        case 1
            xmin1 = X0;
        case 2
            xmin1 = X0 + [d, 0];
        otherwise
            xmin1 = X0 - [d, 0];
    end
    f0 = f(xmin1);
    f_plus = f(xmin1 + [0, d]);
    f_minus = f(xmin1 - [0, d]);
    [fmin, idx] = min([f0, f_plus, f_minus]);
    switch idx
        case 1
            xmin = xmin1;
        case 2
            xmin = xmin1 + [0, d];
        otherwise
            xmin = xmin1 - [0, d];
    end
end
end

```

simplex

```

clc; clear; close all;
f = @(x) (x(1)-2)^2 + (x(2)+3)^2;

```

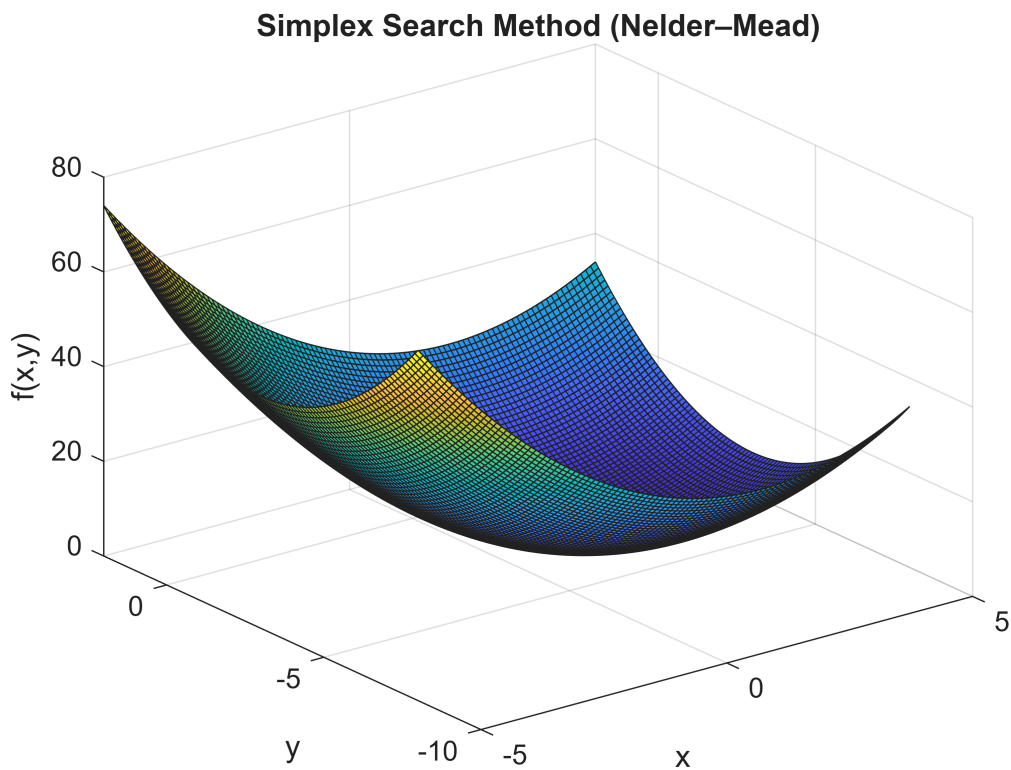
```
x0 = [0 0];
[x_opt, fval, exitflag, output] = fminsearch(f, x0);
fprintf('Optimal point: (%.4f, %.4f)\n', x_opt(1), x_opt(2));
```

Optimal point: (2.0000, -3.0000)

```
fprintf('Minimum function value: %.4f\n', fval);
```

Minimum function value: 0.0000

```
[X,Y] = meshgrid(-5:0.1:5, -8:0.1:2);
Z = (X-2).^2 + (Y+3).^2;
figure;
surf(X,Y,Z); hold on;
plot3(x_opt(1), x_opt(2), fval, 'ro', 'MarkerSize',10, 'MarkerFaceColor','r');
title('Simplex Search Method (Nelder-Mead)');
xlabel('x'); ylabel('y'); zlabel('f(x,y)');
grid on;
```



steepest descent

```
clear all;
clc;
syms x y t
f=2*x^2-2*x*y+y^2;
x0=[1 2];
n=3;
```

```

gradf=gradient(f,[x,y]);
for i=1:n
df=subs(gradf,[x,y], x0);
xt=x0-t*df';
f_xt=subs(f,[x,y], xt);
fdt=diff(f_xt, t);
t1=solve(fdt==0, t);
x0=subs(xt,t, t1);
f0=subs(f,[x,y], x0);
fprintf("X value after %d iteration is:\n", i)
disp(x0);
fprintf(" The corresponding function value is: %.3f \n", f0)
end

```

X value after 1 iteration is:

(1 1)

The corresponding function value is: 1.000

X value after 2 iteration is:

$\left(\frac{1}{2} \ 1\right)$

The corresponding function value is: 0.500

X value after 3 iteration is:

$\left(\frac{1}{2} \ \frac{1}{2}\right)$

The corresponding function value is: 0.250

newtons method

```

clear all;
clc;
syms x y z
k=0;
f=2*x^2-2*x*y+y^2-z-y*z+z^2;
X=[1;2;3];
Df = gradient(f, [x, y, z]);
E = 0.1;
while true
df = double(subs(Df, {x, y, z}, {X(1), X(2), X(3)}));
if norm(df) <= E
break;
end
H = double(subs(hessian(f, [x, y, z]), {x, y, z}, {X(1), X(2), X(3)}));
d = -H \ df;
X = X + d;
k=k+1;
fprintf('Iteration %d: X = [%f, %f, %f], f(X) = %f\n', k, X(1), X(2), X(3),
double(subs(f, {x, y, z}, {X(1), X(2), X(3)})));
end

```

Iteration 1: X = [0.500000, 1.000000, 1.000000], f(X) = -0.500000

```
disp('Converged point:');
```

Converged point:

```
disp(X);
```

```
0.5000  
1.0000  
1.0000
```

```
disp('Function value at the minimum:');
```

Function value at the minimum:

```
disp(double(subs(f, {x, y, z}, {X(1), X(2), X(3)})));
```

```
-0.5000
```